

The Ascent of Autonomous Reasoning: A Comprehensive Analysis of Reinforcement Learning as a Primary Paradigm for Language Model Pre-Training

1. Introduction: The Divergence of Learning and Doing

The contemporary landscape of Artificial Intelligence is dominated by Large Language Models (LLMs) that have achieved remarkable fluency and versatility. However, the foundational methodology used to create these models—supervised pre-training via Maximum Likelihood Estimation (MLE)—is increasingly viewed not as a final destination, but as a transitional technology. For decades, the field has operated under a regime where models are trained to mimic static datasets token by token, a process that fundamentally disconnects the *objective of training* from the *utility of inference*. This report provides an exhaustive analysis of the paradigm shift toward using Reinforcement Learning (RL) not merely as a post-hoc alignment tool, but as the primary engine for pre-training language models from scratch.

The urgency of this shift stems from the inherent limitations of the "Teacher Forcing" paradigm. In standard training, a model predicts the next word based on a ground-truth history provided by the dataset. In deployment, however, the model must predict based on its own self-generated history. This distributional shift, known as *exposure bias*, leads to error accumulation, hallucination, and a lack of long-term planning capabilities. Furthermore, the standard loss function (Cross-Entropy) penalizes any deviation from the specific word chosen in the training set, even if the deviation is semantically valid or superior, effectively punishing the model for creativity and robust reasoning.

Reinforcement Learning offers a principled solution by framing language generation as a sequential decision-making problem. In this view, a Language Model (LM) is an agent, the context is the state, the vocabulary is the action space, and the generation is a trajectory. By training models to optimize for long-term returns—such as logical consistency, semantic coherence, or successful task completion—rather than immediate token mimicry, we unlock the potential for models that do not just statistically approximate human text, but actively reason, plan, and self-correct.

This report explores the theoretical underpinnings, algorithmic innovations, and empirical results of this "RL-First" era. It dissects early hybrid approaches like Scheduled Sampling and MIXER, analyzes the revolutionary "Reinforcement Pre-Training" (RPT) and "Reinforcement

Learning on Pre-Training Data" (RLPT) frameworks, and details the optimization engines like Group Relative Policy Optimization (GRPO) and Decision Transformers that make this scale possible. The analysis posits that we are witnessing the birth of "Self-Correcting Learners"—systems that acquire linguistic intelligence through autonomous exploration and verification, fundamentally redefining the nature of machine learning.

2. The Theoretical Limitations of Maximum Likelihood Estimation

To appreciate the necessity of Reinforcement Learning in pre-training, one must first rigorously deconstruct the failure modes of the dominant Maximum Likelihood Estimation (MLE) paradigm. While MLE has successfully produced models capable of zero-shot generalization, it suffers from structural pathologies that arise from treating sequential generation as a series of independent classification problems.

2.1 The Mathematical Formulation of Teacher Forcing

In the standard autoregressive pre-training setup, a model θ is trained to maximize the likelihood of a sequence $Y = \{y_1, y_2, \dots, y_T\}$ given an input X . The training objective is to minimize the negative log-likelihood (NLL):

$$\mathcal{L}_{MLE}(\theta) = - \sum_{t=1}^T \log P(y_t | y_{1:t-1}, X; \theta)$$

Crucially, during training, the conditioning context $y_{1:t-1}$ is drawn from the *ground truth* data distribution. This is termed "Teacher Forcing".¹ The model is the student, and the dataset is the teacher who corrects the student after every single word, regardless of what the student actually predicted. This ensures stable convergence and efficient parallelization (as seen in the Transformer architecture), but it creates a dependency on a guide that will not exist during deployment.

2.2 Exposure Bias: The Training-Inference Discrepancy

The fundamental flaw of Teacher Forcing is "Exposure Bias." During inference (generation), the ground truth is unavailable. The model must predict y_t conditioned on its own previous predictions $\hat{y}_{1:t-1}$.

- **Distributional Shift:** The distribution of sequences generated by the model $P(\hat{Y})$ inevitably diverges from the data distribution $P(Y^*)$. As the sequence length T

increases, the probability of the model's trajectory remaining within the manifold of the training data decays exponentially.³

- **Error Accumulation:** If the model makes a slight error at step t , the state at $t + 1$ is now slightly "off-distribution." Since the model was never trained to recover from errors (because the teacher always reset the context to the correct path), it lacks a corrective policy. This leads to a compounding cascade of errors, manifesting as repetitive loops, gibberish, or logical derailment in long-form generation.⁵
- **The Kitchen Analogy:** The phenomenon is analogous to learning to cook by watching a master chef. If the student only watches the chef chop vegetables perfectly (Teacher Forcing), they never learn how to salvage a dish if they accidentally chop an onion unevenly. When they are finally alone in the kitchen (inference) and make a mistake, they are helpless.⁷

2.3 The Surrogate Loss Problem

The second critical deficiency is the mismatch between the training objective and the evaluation metrics.

- **Rigidity:** Cross-Entropy is a rigid, token-level metric. It demands exact reconstruction. If the ground truth is "The cat sat on the mat," and the model predicts "The feline rested on the rug," MLE assigns a massive penalty, effectively treating it as entirely wrong as predicting "The car sat on the rug."
- **Global vs. Local:** Human utility is defined by sequence-level properties (coherence, factuality, sentiment, code compilability). MLE optimizes a myopic, step-wise proxy. This is often referred to as the "Surrogate Loss" problem—we optimize Perplexity because it is differentiable, hoping it serves as a good surrogate for quality, but the correlation is often weak.³
- **Sparsity:** In many real-world tasks (reasoning, coding, game playing), the validity of a decision is not known until the end of the sequence. MLE provides a dense signal (a gradient at every token) but essentially "lies" about the importance of intermediate steps, treating "the" and "therefore" as equally critical to the loss function.⁹

Table 1: Structural Comparison of MLE and RL Paradigms

Dimension	Maximum Likelihood Estimation (MLE)	Reinforcement Learning (RL)
Objective	Minimize NLL (Perplexity)	Maximize Expected Return (Utility)
Context Source	Ground Truth (Teacher)	Self-Generated (Policy)

	Forcing)	Rollout)
Error Handling	None (Errors are externally corrected)	Policy learns to recover/adjust
Metric Sensitivity	Rigid (Exact match required)	Flexible (Semantic/Functional validity)
Optimization Scope	Local (Next token only)	Global (Entire sequence trajectory)
Bias	High Exposure Bias	Unbiased (Training matches Inference)

3. The Pre-History of RL Pre-Training: Bridging the Gap

Before the emergence of pure RL pre-training in the mid-2020s, researchers developed several hybrid architectures designed to bridge the gap between MLE and RL. These methods, while often treated as historical footnotes, established the algorithmic primitives—sampling, annealing, and adversarial regularization—that underpin modern RPT.

3.1 Scheduled Sampling: The Curriculum of Independence

One of the earliest attempts to mitigate exposure bias was **Scheduled Sampling**, introduced to gently wean the model off the ground truth.¹⁰

- Mechanism:** The training process involves a coin flip at every time step t . With probability ϵ_i , the model uses the ground truth token y_{t-1} as input (Teacher Forcing). With probability $1 - \epsilon_i$, it uses its own sampled token $\hat{y}_{t-1}$ (Student Forcing).
- Decay Schedules:** The parameter ϵ_i acts as a curriculum controller.
 - Linear Decay:* ϵ_i decreases linearly from 1 to 0 over epochs.
 - Exponential Decay:* $\epsilon_i = k^i$ where $k < 1$.
 - Inverse Sigmoid:* A slower decay that allows for prolonged guidance.
- Impact:** By forcing the model to consume its own outputs during training, Scheduled Sampling exposes the model to its own potential errors, allowing it to learn recovery strategies. However, it was mathematically shown to be inconsistent (the objective function does not asymptotically converge to the true distribution), limiting its utility for

massive foundation models.

3.2 Professor Forcing: Adversarial Regularization

Taking a different approach, **Professor Forcing**¹ utilized the principles of Generative Adversarial Networks (GANs) to align the latent states of the model.

- **The Discriminator:** A discriminator network is trained to distinguish between the hidden states of the RNN when it is running in Teacher Forcing mode versus when it is running in Free-Running (inference) mode.
- **The Generator:** The language model (generator) is trained to "fool" the discriminator. This forces the dynamics of the hidden states to be invariant to the input source.
- **Deep Insight:** If the hidden states look the same regardless of whether the input was a ground truth token or a generated token, then the "drift" caused by exposure bias is effectively neutralized in the latent space.² This method highlighted that the problem of exposure bias could be solved by regularizing the *internal representation* of the model, not just its outputs.

3.3 MIXER: Direct Metric Optimization

The **MIXER (Mixed Incremental Cross-Entropy Reinforce)** algorithm¹¹ was a landmark in directly applying policy gradients to sequence generation.

- **The Challenge:** Metrics like BLEU and ROUGE are non-differentiable. One cannot compute $\frac{\partial \text{BLEU}}{\partial \theta}$.
- **The REINFORCE Solution:** MIXER uses the REINFORCE algorithm (a Monte Carlo policy gradient method) to estimate the gradient. The "reward" is the BLEU score of the generated sequence.
- **Annealing Strategy:** Recognizing that RL is sample-inefficient and struggles to learn from scratch (the "Cold Start" problem), MIXER starts with MLE training. It then gradually introduces the RL objective, starting from the last token and moving backward.
 - Epochs $1-N$: Pure MLE.
 - Epochs $N+1$: MLE for steps 1 to $T-1$, RL for step T .
 - Epochs $N+k$: MLE for steps 1 to $T-k$, RL for steps $T-k$ to T .
- **Significance:** MIXER demonstrated that mixing supervised priors with RL fine-tuning was feasible, a strategy that would later evolve into the standard "Pre-train then Align" pipeline, though MIXER aimed to do this during the primary training phase itself.

4. Reinforcement Learning on Pre-Training Data (RLPT): The Modern Paradigm

The most significant advancement in this domain, solidified by research in 2024-2025, is the

concept of **Reinforcement Pre-Training (RPT)** or **Reinforcement Learning on Pre-Training Data (RLPT)**. This paradigm posits that RL should not be reserved for fine-tuning on human preferences, but should be the primary engine for learning the structure of language itself from unlabeled corpora.¹²

4.1 Reframing Pre-Training: From Prediction to Reasoning

Standard pre-training treats text as a stream of data. RLPT treats text as a *problem solving environment*. The core innovation is the shift from "Next-Token Prediction" (NTP) to "Next-Segment Reasoning".¹²

- **The Reasoning Trace:** Instead of predicting just the next word, the model is tasked with generating a "reasoning trace"—a coherent segment of text (e.g., a sentence, a paragraph, or a logical step) that bridges the current context to the future context.
- **The Environment:** The "environment" is the vast corpus of the internet. The "state" is the text read so far. The "goal" is to reconstruct or predict the future text.
- **The Reward:** Unlike RLHF which requires a human-trained reward model, RLPT derives its reward *intrinsically* from the data. If the model's generated segment semantically aligns with the actual next segment in the corpus, it receives a positive reward.

4.2 The RLPT Training Loop and Architectures

The RLPT process¹³ involves a distinct cycle of exploration and verification that differs significantly from the SGD updates of MLE.

1. **State Observation ($s_{<i}$):** The model reads the prefix context from the corpus.
2. **Policy Rollout (a_i):** The model generates a candidate segment $\hat{s}_i$. This generation is stochastic, encouraging exploration of different phrasings or logical paths.
3. **Reward Calculation (T_i):**
 - The generated segment $\hat{s}_i$ is compared to the ground truth segment s_i .
 - *Naive approach:* Exact string matching (Sparse, problematic).
 - *RLPT approach: Semantic Consistency Verification.* A generative reward model (G_{rm}) or a heuristic function checks if $\hat{s}_i$ is a valid semantic prefix of the future text.

The reward is often binary ($r \in \{0, 1\}$) or graded based on semantic overlap (e.g., embedding similarity).¹⁵
4. **Policy Update:** The model updates its parameters to maximize the expected reward using algorithms like GRPO or PPO.

4.3 Autoregressive vs. Middle Segment Reasoning

RLPT employs two distinct tasks to cultivate different cognitive capabilities in the model¹⁵:

- **Autoregressive Segment Reasoning (ASR):** Given context A , predict the next segment B . This reinforces causal reasoning and forward planning. It mimics standard language modeling but at a trajectory level, forcing the model to maintain coherence over longer horizons.
- **Middle Segment Reasoning (MSR):** Given context A and future context C , infer the bridging segment B . This is a "fill-in-the-blank" task at the paragraph level. It requires constraint satisfaction and bidirectional reasoning, effectively teaching the model to plan "backwards" from a goal (context C) and "forwards" from a start (context A).

4.4 Scaling Laws: Compute as the New Data

Perhaps the most profound finding in RPT research is the emergence of new scaling laws.

- **MLE Scaling:** Performance $\propto$ Parameters $\times$ Dataset Size.
- **RLPT Scaling:** Performance $\propto$ Training Compute (Number of Rollouts).¹²
- **Implication:** This suggests that we can improve model performance not just by collecting more data (which is becoming scarce), but by letting the model "think" more about the data it already has. By exploring multiple trajectories for every segment of text, the model extracts richer signals and learns more robust representations than simple memorization allows. RPT models show favorable scaling curves where increased training compute consistently yields better reasoning capabilities, even on fixed datasets.¹⁶

5. Group Relative Policy Optimization (GRPO): The Engine of Scale

The theoretical elegance of RL pre-training collides with a brutal reality: Reinforcement Learning is computationally expensive. Standard algorithms like Proximal Policy Optimization (PPO) require maintaining a **Critic** (Value Function) network that is often as large as the **Actor** (Policy) network. For a 70B parameter model, this doubling of memory requirements is prohibitive for pre-training. **Group Relative Policy Optimization (GRPO)** has emerged as the critical enabler for scaling RL to the foundation model level.¹⁷

5.1 The Memory Bottleneck of PPO

In PPO, the advantage function $A(s, a)$ is estimated as:

$$A(s, a) = Q(s, a) - V(s)$$

where $V(s)$ is the value function estimated by the Critic network. The Critic must be trained alongside the Actor to predict the expected future reward. This requires:

1. Forward/Backward passes for the Actor.
2. Forward/Backward passes for the Critic.
3. Storage of gradients and optimizer states for both.

This effectively doubles the VRAM usage and compute cost compared to supervised learning.

5.2 The GRPO Mechanism: Relative Advantage

GRPO eliminates the Critic entirely. Instead of estimating an absolute value $V(s)$, it estimates a **relative baseline** by sampling a group of outputs for the same input.¹⁹

- **Group Sampling:** For a given input prompt q , the model generates a group of G outputs $\{o_1, o_2, \dots, o_G\}$ sampled from its current policy π_θ .
- **Group Evaluation:** Each output receives a reward r_i from the environment/reward model.
- **Relative Advantage Estimation:** The advantage for output o_i is calculated by normalizing its reward against the group statistics:

$$A_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\}) + \epsilon}$$

- **Intuition:** If an output o_i scores better than the average of its "siblings" generated from the same state, it is a good action ($A_i > 0$) and should be reinforced. If it scores worse, it should be discouraged. This self-referential baseline removes the need for a learned Value Function.

5.3 The Objective Function and KL Penalty

The GRPO objective incorporates the standard PPO clipping mechanism to prevent destructive updates, along with a KL-divergence penalty to ensure the model does not drift too far from a reference distribution (usually the model from the previous epoch).¹⁹

$$\mathcal{L}_{\text{GRPO}}(\theta) = \mathbb{E} \left[\right]$$

This formulation allows for massive parallelization. Since the baseline is computed on the fly from the batch, there is no need to train or sync a separate Critic model. DeepSeekMath demonstrated that GRPO is not only more efficient but often outperforms PPO in mathematical reasoning tasks by reducing the variance associated with Value Function

estimation errors.¹⁸

Table 2: Comparative Analysis of RL Optimization Algorithms for LLMs

Feature	PPO (Proximal Policy Optimization)	GRPO (Group Relative Policy Optimization)
Critic Requirement	Yes (Must be trained/maintained)	No (Critic-free)
Advantage Estimation	$A = R -$ (Model-based)	$A = (r -$ (Sample-based)
Memory Footprint	Very High (2x Model Size)	Low (~1x Model Size)
Compute Efficiency	Lower (Dual backward passes)	Higher (Single backward pass)
Stability	Dependent on Critic convergence	Robust (Self-normalizing)
Primary Use Case	RLHF (Fine-tuning)	RL Pre-Training / Reasoning

6. Offline Reinforcement Learning: The Decision Transformer

Parallel to the development of online RL methods like GRPO, a separate lineage of research has reimagined RL as a sequence modeling problem, leading to the **Decision Transformer (DT)** architecture. This approach is particularly relevant for "from scratch" training because it allows RL objectives to be integrated directly into the standard Transformer architecture without the complexity of gradients and rewards during the training loop itself.²¹

6.1 RL as Conditional Sequence Generation

The central insight of the Decision Transformer is that an RL trajectory

$\tau = (s_1, a_1, r_1, s_2, a_2, r_2, \dots)$ can be modeled just like a sentence of text.

- **The Paradigm Shift:** Instead of using Bellman updates or value functions to find the optimal policy, DT treats the optimal policy as a conditional generation task: "Given the

current state and a desire for high reward, what is the next action?"

- **The Input Sequence:** The model is fed a sequence of (Return-to-Go, State, Action). The "Return-to-Go" ($\hat{R}_t$) is the sum of future rewards expected from timestep t .

$$\tau = (\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots)$$

- **Inference:** To control the agent, one simply "prompts" the model with a high Desired Return (e.g., $\hat{R}_1 = 100$). The autoregressive mechanism then generates the actions that are statistically most likely to result in that return, based on the patterns seen in the training data.²¹

6.2 Application to Language Modeling

In the context of language, the **Trajectory Transformer** applies this logic to text.²⁴

- **State:** The context (prompt).
- **Action:** The next token (or segment).
- **Reward:** A quality metric (e.g., semantic coherence, toxicity score, or user engagement signal).
- **Training:** The model is trained on a massive dataset of text, where each sequence is annotated with its quality score. It learns the relationship between "quality" and "text patterns."
- **Generation:** During generation, the model is conditioned on the token <High_Quality>, forcing it to sample from the subset of its learned distribution that corresponds to high-reward trajectories.²⁶ This effectively allows for "RL training" on offline data without any active interaction with an environment, making it a powerful technique for pre-training on static web corpora that have been scored for quality.

6.3 Online Decision Transformers (ODT)

While standard DTs are offline learners (limited by the quality of the dataset), the **Online Decision Transformer (ODT)** incorporates an exploration loop.

- **Active Learning:** The ODT interacts with the environment (or simulator) to generate *new* trajectories. These new experiences are added to the replay buffer, and the model is updated.
- **Relevance to Pre-Training:** For language models, ODT represents a bridge between static pre-training and active RL. An ODT-LM can start by learning from a static corpus (offline), and then switch to an "online" phase where it generates text, receives rewards (from a verifier or heuristic), and updates its internal map of "actions that lead to high rewards".²⁷

7. The Action Space Challenge: Navigating the

Vocabulary

Training language models from scratch with RL presents a unique challenge that does not exist in robotics or game playing: the magnitude of the action space. In a typical Atari game, the action space size $|A|$ is roughly 18. In a modern LLM, $|A|$ is the vocabulary size, typically 50,000 to 128,000 tokens.

7.1 The Failure of Value-Based Methods (DQN)

Deep Q-Networks (DQN) require computing the Q-value for every possible action in a given state: $Q(s, a) \forall a \in A$.

- **Computational Prohibitive:** Calculating and storing 100,000 Q-values at every step of generation is computationally infeasible.
- **Overestimation Bias:** With such a massive number of actions, the probability of the max operator selecting an action with an overestimated Q-value due to noise approaches 1. This leads to severe training instability.²⁹
- **Sparsity:** Most tokens are irrelevant for any given context. The Q-values for 99.9% of the vocabulary should be effectively zero (or negative infinity), but learning this sparse landscape from scratch is incredibly sample-inefficient.³⁰

7.2 Stochastic Q-Learning and Action Clustering

To overcome this, researchers have developed specialized techniques for large discrete action spaces.

- **Stochastic Q-Learning:** Instead of maximizing over the entire vocabulary, this algorithm samples a small subset of actions (e.g., $k = 100$) and performs the max operation only over this subset. Theoretical analysis shows that with sufficient sampling, this converges to the optimal policy with significantly lower compute.²⁹
- **Action Clustering:** Techniques like K-Means clustering are used to group semantically similar tokens into "skills" or "super-actions." The RL agent first selects a cluster, and then a lower-level policy selects the specific token. This hierarchical decomposition reduces the effective dimensionality of the action space from 50,000 to a manageable number of clusters (e.g., 500).³¹

7.3 Why Policy Gradients Win

The difficulty of value estimation in high-dimensional spaces explains why **Policy Gradient** methods (REINFORCE, PPO, GRPO) are the dominant choice for Language RL. Policy

gradients operate directly on the probability distribution $\pi(a|s)$. Since the LLM already

outputs a valid probability distribution (via the Softmax layer), policy gradients can optimize the likelihood of the *sampled* action without needing to estimate the value of all *unchosen* actions. This aligns perfectly with the architecture of the Transformer, making Policy Gradients the natural fit for text generation.³²

8. The Exploration-Exploitation Dilemma in Language

In "from scratch" RL training, the model must balance **Exploration** (trying new words/phrases to find better ways to express an idea) and **Exploitation** (using the words it knows will yield a reward).

8.1 The "Learning Cliff" and Cold Starts

A purely random initialized model will generate gibberish. If the reward function is "Is this a valid English sentence?", a random model will receive a reward of 0 for millions of steps. This is known as the "Learning Cliff" or the sparse reward problem.

- **The Cold Start Phase:** Almost all successful "RL from scratch" implementations (including Microsoft's RPT) acknowledge the need for a **Cold Start**. This is a brief period of Supervised Fine-Tuning (SFT) or MLE pre-training on a small, high-quality dataset. This "primes" the model to a minimum level of capability where it can generate semi-coherent text and start receiving non-zero rewards.¹³
- **Scaffolded Rewards:** To mitigate the cliff, "Scaf-GRPO" introduces scaffolded rewards. Instead of giving a reward only for the final answer, the model is rewarded for intermediate steps or for using correct templates/structures. This provides a "trail of breadcrumbs" that guides the random model toward the region of high reward.³³

8.2 Entropy Regularization

To prevent the model from collapsing into a single, repetitive output (mode collapse), RL objectives in language typically include an **Entropy Bonus**.

$$L = L_{policy} + \beta H(\pi(s))$$

Maximizing entropy $H(\pi)$ encourages the model to keep the probability distribution flat, ensuring it continues to explore diverse vocabulary and phrasing. This is crucial for preventing the "gaming" of the reward function, where the model finds one generic sentence (e.g., "I don't know") that is technically safe/valid and repeats it infinitely.¹⁹

8.3 The Gumbel-Softmax Relaxation

One of the technical barriers to training sequence models with RL is that token selection is a discrete, non-differentiable operation. We cannot backpropagate through `argmax` or `multinomial_sample`.

- **The Trick:** The **Gumbel-Softmax** distribution³⁴ provides a continuous approximation of the categorical distribution. It allows the model to sample tokens in a way that is differentiable with respect to the model parameters.
- **Straight-Through Estimator:** In the forward pass, the model samples a discrete token (to maintain valid text generation). In the backward pass, the gradient flows through the continuous Gumbel approximation. This technique, often used in SeqGANs and variations of RPT, allows for lower-variance gradient estimation than pure REINFORCE.³⁶

9. Case Studies and Empirical Benchmarks

The theoretical constructs discussed above have been validated in several landmark studies that prove the viability of RL as a primary training paradigm.

9.1 Microsoft's Reinforcement Pre-Training (2025)

The RPT study¹² represents the state-of-the-art in this field.

- **Experiment:** The researchers compared a model trained via standard MLE against one trained via RPT (using interleaved ASR/MSR tasks and GRPO).
- **Key Result:** The RPT model achieved significantly lower perplexity and higher accuracy on downstream reasoning tasks (MMLU, GSM8K).
- **Insight:** The study showed that "Next-Token Reasoning" (optimizing the path) yields better representations than "Next-Token Prediction" (optimizing the target). The RPT model learned to "think ahead," evidenced by its superior performance on tasks requiring multi-step logic.

9.2 DeepSeekMath and GRPO

While DeepSeekMath¹⁸ utilized a base coder model, its use of GRPO to enhance mathematical reasoning is a powerful case study for RL's generative capabilities.

- **Discovery of Novel Solutions:** The RL-trained model was observed to generate correct mathematical proofs using pathways and logical leaps that were *not* present in the supervised training data. This confirms that RL allows the model to explore the solution space and discover novel, optimized policies for reasoning, effectively creating new knowledge rather than just retrieving it.

9.3 Trajectory Transformers in Control

Although primarily a control/robotics paper, the Trajectory Transformer²⁴ proved that the Transformer architecture is a general-purpose sequence modeler. By successfully training a Transformer to control a robot using the same architecture used for GPT, it demonstrated that the distinction between "Language" and "Action" is artificial. This paved the way for treating language generation as a control problem, validating the core thesis of RLPT.

10. Future Directions: The Era of Self-Correcting Intelligence

The transition from MLE to RL-First pre-training marks a maturation point in the history of Artificial Intelligence. We are moving from the era of **Static Pattern Matchers** to the era of **Dynamic Reasoners**.

10.1 The End of Hallucination?

Hallucination is largely a symptom of the exposure bias and lack of grounding in MLE models. RL pre-training addresses this root cause. By training models to verify their own segments (via RLPT) and punishing trajectories that lead to semantic inconsistencies, we are effectively engineering "truth-seeking" as a fundamental objective of the neural network's optimization landscape.

10.2 Autonomous World Models

Future iterations of RPT will likely integrate more complex "Simulators" or "World Models" into the reward loop. Instead of just checking for semantic consistency with the next text segment, the model might be rewarded for generating code that compiles, proofs that verify in a formal theorem prover, or hypotheses that align with a knowledge graph. This effectively turns the training process into a continuous scientific method: Hypothesize (Generate) → Test (Verify) → Learn (Update).

10.3 Conclusion

Reinforcement Learning is no longer just a fine-tuning varnish applied to a supervised base. It is becoming the foundation itself. By framing language modeling as a sequential decision-making problem, employing scalable optimization algorithms like GRPO, and leveraging the intrinsic structure of data through objectives like Next-Segment Reasoning, we are building models that do not just speak, but think. This paradigm shift promises to deliver agents that are more robust, more accurate, and capable of the kind of long-horizon planning that has long eluded the grasp of statistical learning. The "Kitchen" of AI training is finally changing—the student is no longer just watching the master; they are learning to cook.

Works cited

1. Professor Forcing: A New Algorithm for Training Recurrent Networks - NIPS, accessed January 29, 2026, <http://papers.neurips.cc/paper/6099-professor-forcing-a-new-algorithm-for-training-recurrent-networks.pdf>
2. Teacher forcing - Wikipedia, accessed January 29, 2026, https://en.wikipedia.org/wiki/Teacher_forcing

3. An Understanding of Learning from Demonstrations for Neural Text Generation - The ICLR Blog Track, accessed January 29, 2026, <https://iclr-blog-track.github.io/2022/03/25/text-gen-via-lfd/>
4. Teacher Forcing Recovers Reward Functions for Text Generation - arXiv, accessed January 29, 2026, <https://arxiv.org/pdf/2210.08708>
5. Exposure Bias in Machine Learning - Emergent Mind, accessed January 29, 2026, <https://www.emergentmind.com/topics/exposure-bias>
6. Why Exposure Bias Matters: An Imitation Learning Perspective of Error Accumulation in Language Generation - ACL Anthology, accessed January 29, 2026, <https://aclanthology.org/2022.findings-acl.58.pdf>
7. Understanding Exposure Bias in Generative Models | by Shubham Mishra - Medium, accessed January 29, 2026, https://medium.com/@shubham_3306/understanding-exposure-bias-in-generative-models-857ff7509062
8. LLM Evaluation Is Broken: Why BLEU and ROUGE Don't Measure Real Understanding, accessed January 29, 2026, <https://pub.towardsai.net/llm-evaluation-is-broken-why-bleu-and-rouge-dont-measure-real-understanding-8f3b167f7931>
9. [2204.07696] Efficient Reinforcement Learning for Unsupervised Controlled Text Generation, accessed January 29, 2026, <https://arxiv.org/abs/2204.07696>
10. Scheduled Sampling for Sequence Prediction with Recurrent Neural ..., accessed January 29, 2026, <https://arxiv.org/abs/1506.03099>
11. Sequence Level Training with Recurrent Neural Networks, accessed January 29, 2026, <https://arxiv.org/abs/1511.06732>
12. Reinforcement Pre-Training - arXiv, accessed January 29, 2026, <https://arxiv.org/html/2506.08007v1>
13. Reinforcement Learning on Pre-Training Data - arXiv, accessed January 29, 2026, <https://arxiv.org/html/2509.19249v1>
14. Reinforcement Pre-Training (RPT) By Microsoft Explained - YouTube, accessed January 29, 2026, https://www.youtube.com/watch?v=mvdJ0VXu_2g&v1=en
15. [2509.19249] Reinforcement Learning on Pre-Training Data - arXiv, accessed January 29, 2026, <https://arxiv.org/abs/2509.19249>
16. [2506.08007] Reinforcement Pre-Training - arXiv, accessed January 29, 2026, <https://arxiv.org/abs/2506.08007>
17. What is GRPO? Group Relative Policy Optimization Explained - DataCamp, accessed January 29, 2026, <https://www.datacamp.com/blog/what-is-grpo-group-relative-policy-optimization>
18. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models - arXiv, accessed January 29, 2026, <https://arxiv.org/pdf/2402.03300>
19. A vision researcher's guide to some RL stuff: PPO & GRPO - Yuge (Jimmy) Shi, accessed January 29, 2026, <https://yugeten.github.io/posts/2025/01/ppogrpo/>
20. Splitting open RL post training-Part 3, accessed January 29, 2026, <https://kaifkohari10.medium.com/splitting-open-rl-post-training-part-3-b4e0aa78d945>

21. Stanford CS25: V1 I Decision Transformer: Reinforcement Learning via Sequence Modeling, accessed January 29, 2026, <https://www.youtube.com/watch?v=w4Bw8WYL8Ps>
22. [2106.01345] Decision Transformer: Reinforcement Learning via Sequence Modeling - arXiv, accessed January 29, 2026, <https://arxiv.org/abs/2106.01345>
23. opendilab/awesome-decision-transformer - GitHub, accessed January 29, 2026, <https://github.com/opendilab/awesome-decision-transformer>
24. Sequence Modeling Solutions for Reinforcement Learning Problems - Berkeley AI Research, accessed January 29, 2026, <https://bair.berkeley.edu/blog/2021/11/19/trajectory-transformer/>
25. A Comparison Between Decision Transformers and Traditional Offline Reinforcement Learning Algorithms in Varying Reward Settings for Continuous Control Tasks - arXiv, accessed January 29, 2026, <https://arxiv.org/html/2511.16475v1>
26. Pre-trained Language Models Improve the Few-shot Prompt Ability of Decision Transformer, accessed January 29, 2026, <https://arxiv.org/html/2408.01402v2>
27. DODT: Enhanced Online Decision Transformer Learning through Dreamer's Actor-Critic Trajectory Forecasting - arXiv, accessed January 29, 2026, <https://arxiv.org/html/2410.11359v1>
28. Safety considerations for online generative modeling - AI Alignment Forum, accessed January 29, 2026, <https://www.alignmentforum.org/posts/BMfNu82iunjQKyQA9/safety-consideration-s-for-online-generative-modeling>
29. fouratifaes/stochdqn: Stochastic Q-learning for Large Discrete Action Spaces - GitHub, accessed January 29, 2026, <https://github.com/fouratifaes/stochdqn>
30. Deep Q-Learning for Text Generation - Stanford University, accessed January 29, 2026, <https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1234/final-reports/final-report-169687355.pdf>
31. Subwords as Skills: Tokenization for Sparse-Reward Reinforcement Learning - NIPS, accessed January 29, 2026, https://proceedings.neurips.cc/paper_files/paper/2024/file/7d0c7a899224cd178b2e0cbe39b5a5-Paper-Conference.pdf
32. Efficient (Soft) Q-Learning for Text Generation with Limited Good Data - ACL Anthology, accessed January 29, 2026, <https://aclanthology.org/2022.findings-emnlp.518.pdf>
33. [2510.19807] Scaf-GRPO: Scaffolded Group Relative Policy Optimization for Enhancing LLM Reasoning - arXiv, accessed January 29, 2026, <https://arxiv.org/abs/2510.19807>
34. Gumbel-Softmax: Differentiable Discrete Sampling - Emergent Mind, accessed January 29, 2026, <https://www.emergentmind.com/topics/gumbel-softmax>
35. What is Gumbel-Softmax?. A differentiable approximation to... | by Wanshun Wong | TDS Archive | Medium, accessed January 29, 2026, <https://medium.com/data-science/what-is-gumbel-softmax-7f6d9cdcb90e>
36. GANS for Sequences of Discrete Elements with the Gumbel-softmax Distribution

- arXiv, accessed January 29, 2026, <https://arxiv.org/abs/1611.04051>
- 37. Synthetic Data with Gumbel-Softmax Activations - AI Infrastructure Alliance, accessed January 29, 2026, <https://ai-infrastructure.org/synthetic-data-with-gumbel-softmax-activations/>